

自掏腰包一万元，拥抱AI这一年，我的工具、实践和思考

原创 李是希 腾讯云开发者 2026年4月10日 08:46 北京

关注腾讯云开发者，一手技术干货提前解锁👉



腾讯云开发者

腾讯云官方社区公众号，汇聚技术开发者群体，分享技术干货，打造技术影响力交...
1095篇原创内容

公众号

我应该是比较早就开始关注大模型，并且也蛮愿意积极拥抱 AI 的同学

1. 产品体验方面，付费使用过的产品就包括但不限于 claude code max、gpt pro/plus、gemini pro、perplexity、cursor、lovable、gamma、windsurf、bolt.new、replit、seedance2.0、...
(至今已经撒币超万元)。
2. 开发实践方面，从早期探索性的 AutoGPT、BabyAGI，到 LangChain/LangGraph 等编排框架和 Dify 等低代码平台的兴起，再到如今逐渐弱化框架，以 Agent + Skill/Tool 来完成复杂任务的新范式。我也一直保持跟进和钻研，并且很开心能够在 AI 导向的团队里面有所应用。

坦白讲，拥抱 AI 的这一路过程并不是那么顺利。因为每个时期的 AI 都有着不同的特点，很多时候当我费尽心思去钻研，自以为有点弄明白 AI 应该怎么玩的时候，基座模型和行业范式常常就会迎来颠覆式的变革，很多过往的经验都会倾覆，然后只能从头再来。但是在经历这种不停摧毁和重建的过程中，我认为还是沉淀了一些很有意义的东西值得和大家进行分享。

本文主要分为两部分进行撰写

第一部分偏务实，主要介绍一些可以直接落地的工具、实践和方法论。虽然更多是适配我个人使用 AI 的场景，但希望也能够对大家有所启发，当然更欢迎提出意见一起优化方案。

第二部分偏务虚，主要分享一些我个人使用 AI 的碎碎念，以及对于 AI 时代的一些思考和暴论。

01

工具、实践和方法论

就像前文说的那样，在使用 AI 的过程中我经历了很多次的摧毁和重建。我经常在想，我往后的哪些努力能够不会被太轻易的被淹没在 AI 迭代的时代浪潮里，或者说即使终将被淹没，哪些学习和实践能够让我在当下得到更大的收益。以下是我认为最重要的三个方面，依次进行展开。

1.1 把 Agent 用起来

1.1.1 MAC 工具链分享

在大模型时代以前，因为个人并行执行的任务有限，往往最多三个屏幕就可以基本涵盖大部分工作所需的应用（IDE、浏览器、命令行、IM 等）。但是现如今我们有能力进行多个需求/项目的同时开发，以及可能同时使用非常多不同的 AI 工具，那么传统的桌面配置开始变得捉襟见肘。

在这个背景下，我对 MAC 电脑的工具链进行了一次迭代，比较核心的组件如下



各个工具详情

工具	性质	主要能力
Raycast	闭源免费	我用到比较多的功能是「应用快捷键启动」、「剪贴板历史」
AeroSpace	开源	支持窗口自动分屏、工作区管理、应用自动归位等功能
Ghostty	开源	多位知名 AI 技术大咖推荐的命令行客户端
Yazi	开源	【命令行工具】TUI 三栏文件浏览器、文件/图片预览

lazygit	开源	【命令行工具】命令行进行 git 相关操作
bttop	开源	【命令行工具】CPU/内存/磁盘/网络/进程实时监控
fzf	开源	【命令行工具】历史命令搜索、文件搜索、目录搜索
Claude Code、CodeBuddy 等	闭源付费	AI 编码、Hooks 自动化、Skills 技能库、审计和执行
Cockpit	自研	跨机器任务状态仪表盘、Hooks 驱动生命周期自动同步
tmux	开源	远程终端复用、会话持久化、断线恢复

其中比较值得一提的是这两点

1. 最推荐的工具是 aerospace 和 raycast，因为很多人可能不是像我一样切换到高频使用命令行，但是「管理/切换工作区」、「快捷键打开应用」、「剪贴板历史记录」的诉求或多或少都是存在的，属于很好用的老牌工具。
2. 这里面提到的 Cockpit 是我个人开发实现的工具。比如我可能经常会有多个 Agent 任务分别在“windows 远程机器、linux 远程机器、本地、虚拟机”同时开展，这个工具的目的是希望能够即时关注到各个 Agent 的状态，避免有单个 Agent 因为“等待审核”、“异常报错”等情况长时间闲置（因避免暴露工作项目内容，下图中任务名称系替换后进行的展示）。



但目前还是存在一些偶现不稳定的问题，以及暂时还没有良好支持 claude code 以外的 Agent 工具。如果大家有更好的方案也欢迎探讨和指教，这种日常刚需的组件我一直试图探寻更稳定的解决方案，而不是依赖 AI 持续迭代。

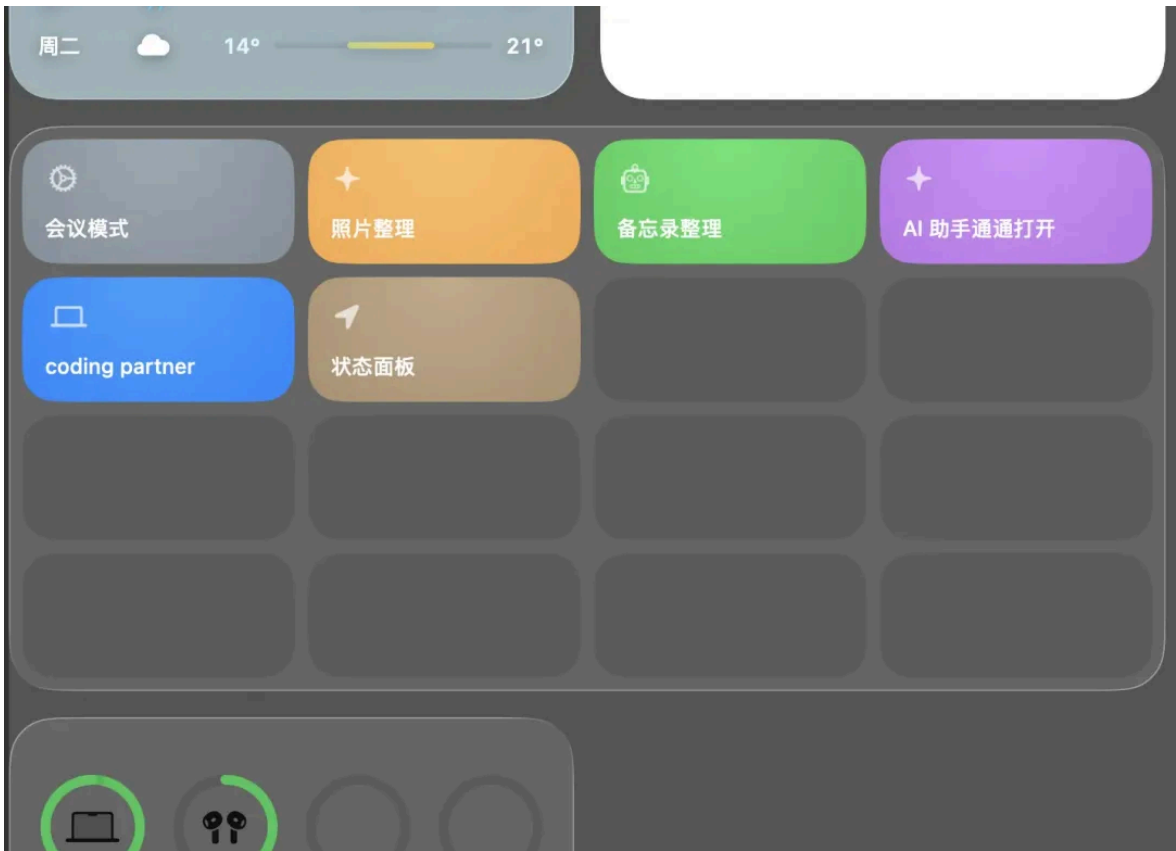
1.1.2 一起来吃低垂的果实

在大模型时代来临以后，对于我们所有人最直接的收益是「我们能比较轻松做到一些以前比较麻烦或者完全做不了的事情」。比如最简单就可以通过 openclaw、workbuddy、claude code 等各种工具去驱动海量的 skill 库，以及在此基础上形成自己的 AI 协作模式。我觉得是很值得多去尝试的，一方面是通过这些小的实践能够给我们在 AI 时代的焦虑环境中多带来一些正反馈，另一方面也是能够更好的认知 AI 的边界在哪里，方便拓展应用到项目或者生活。

给大家分享几个我做过的比较有意义的实践

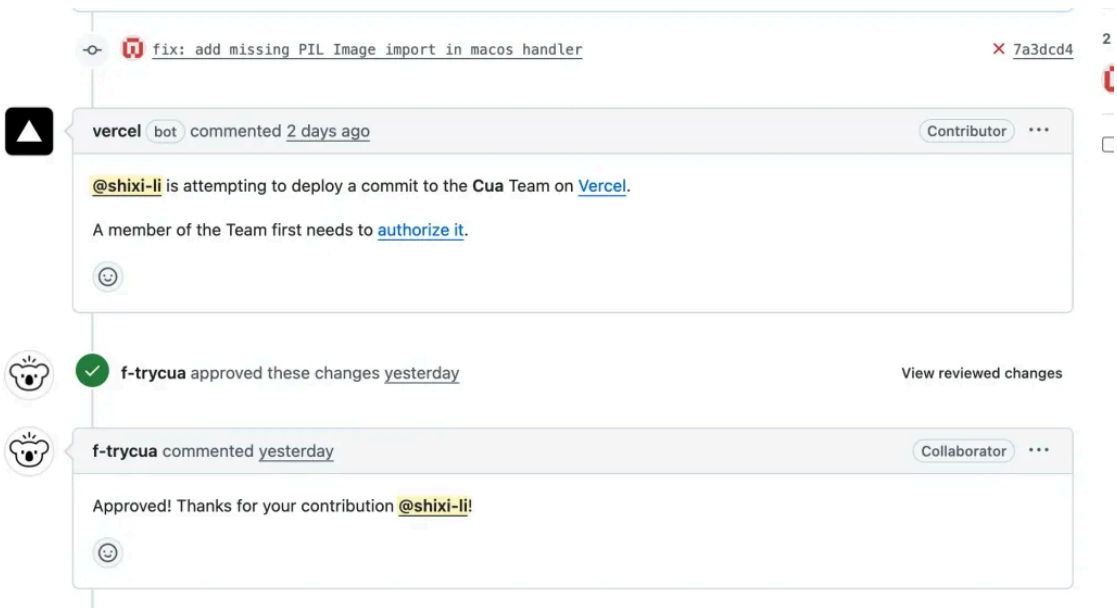
快捷指令

我是在古早时期就比较热衷捣鼓 ios 快捷指令的人，但是因为苹果生态更新频繁所以导致快捷指令时常不稳定，以前做这件事非常吃力不讨好。但是现在完全可以让大模型自己去开发快捷指令轻易实现各种各样的事情，虽然仍然不稳定，但是有我的 AI 小伙伴可以帮我在需要的时候持续维护~



顺手提点 PR

我们开发过程中经常会使用各种开源项目，以前即使发现开源项目中存在需要优化的地方，很多时候因为工作繁忙，确实没有精力再去为社区做贡献。可现在因为 github 对于命令行鉴权的良好支持，遇到疑似问题以后，完全可以让 llm 自主的完成优化流程并提出 PR。



还可以打 Kaggle 比赛

比如在 kaggle 打比赛往往是一件很耗精神的事情，需要机智的脑袋疯狂转，在工作以后很难再有充足的时间去折腾。春节期间我是在外地旅游，就远程让我的台式机基于 claude code 去分析比赛、训练模型、提交数据，大概每过一两天我再登上去瞅一眼，和 AI 小伙伴沟通半个小时。然后就这么全自动托管着，约 4000 支队伍的比赛最高取得过在榜第六的成绩（当然后面掉下来了 -.-||）

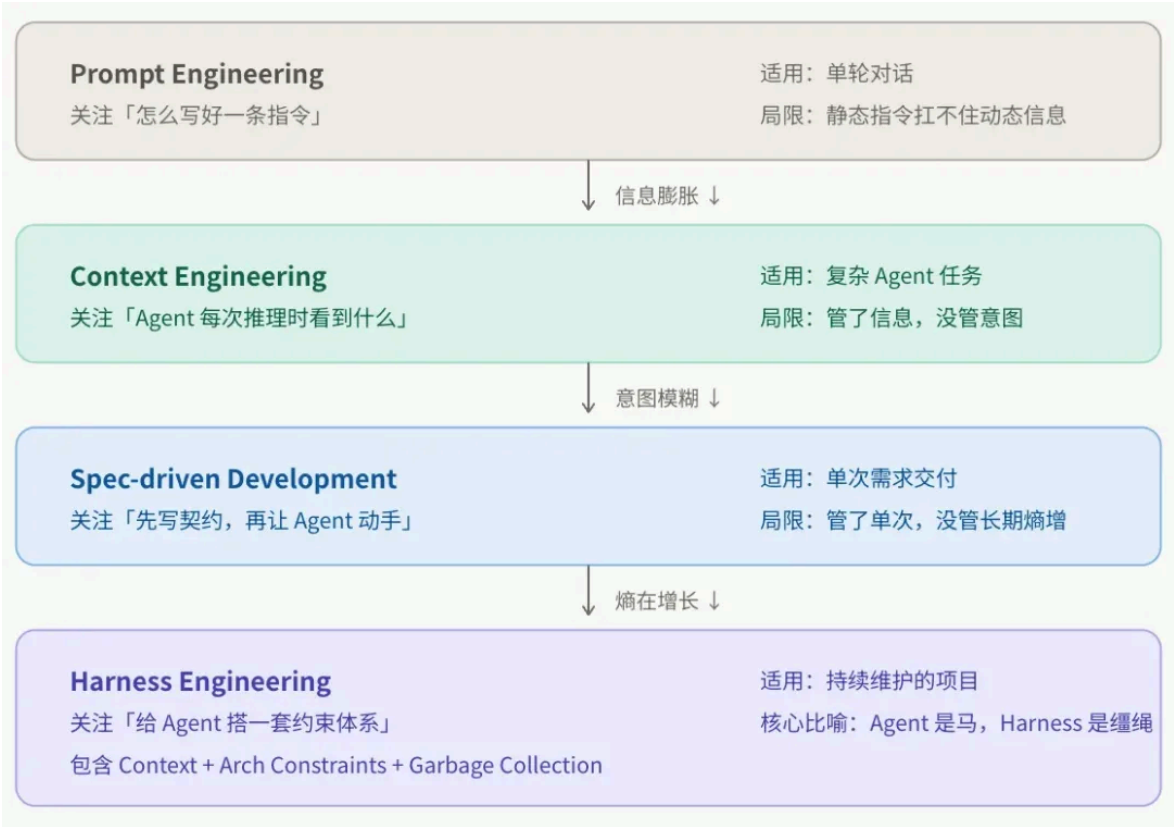
#	Team	Members	Score	Entries
1	Pirhosseinlou		0.95413	39
2	Zbigniew Kirchner		0.95407	156
3	Sung Heo		0.95407	164
4	Chris Deotte		0.95407	91
5	kava1		0.95406	155
6	lishixi		0.95406	55



Your Best Entry!
Your most recent submission scored 0.95406, which is the same as your previous score. Keep trying!

1.2 从 Prompt Engineering 到 Harness Engineering

上一节我们主要聊了如何把越来越多的 AI 能力丝滑用起来，这里我们就继续探讨如何能够把 Agent 用的好一点，也就是来到了老生常谈的提示词问题。我认为大模型应用的本质仍然是提示词的撰写、组织和管理。过去一年多，被热议的 AI 范式大概是沿着四个阶段递进演化，每一个阶段其实都在解决前一个阶段的核心问题。



当然这不是一个严格的时间线，这些阶段之间本身也是互相包含的，甚至很多和 AI 契合度高的同学可能最开始就是或多或少在践行着「Harness Engineering」的理念，本节尝试依次做出介绍和分析。

1.2.1 Prompt Engineering：来时的路

这个大家应该都比较有体感。最开始我们跟大模型打交道的方式就是写 prompt，怎么措辞、怎么给例子（few shot）、怎么引导思维链，社区里也出现了大量的 prompt 技巧分享。这些在单轮对话场景下确实有效。

但问题是，当我们开始用 Agent 去执行一些稍微复杂一点的任务，比如跨文件的代码重构、多步骤的需求实现，单靠一个写得再好的 prompt 也很难稳定产出高质量结果。因为 Agent 在长时运行中面对的信息量是动态膨胀的，对话历史越来越长、工具调用的结果在不停堆积、外部上下文在变化，一个静态的 prompt 根本 hold 不住这些。

这也是为什么业界开始从 Prompt Engineering 转向一个新概念：Context Engineering。

1.2.2 Context Engineering: 管好 Agent 看到的一切

Context Engineering 这个词在 2025 年中开始被大量讨论，Shopify CEO Tobi Lütke 在 X 上的一条帖子 (<https://x.com/tobi/status/1935533422589399127>) 让它出圈，Anthropic 也专门发文 Effective context engineering for AI agents (<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>) 阐述。

简单来说，Prompt Engineering 关注的是"怎么写好指令"，而 Context Engineering 关注的是"Agent 每次推理时，整个信息环境长什么样"。这个信息环境包括系统指令、可用的工具定义、外部数据、对话历史、memory 等等，这些东西加在一起，才是 Agent 真正"看到"的全部。

核心思想其实很朴素：让 Agent 在每一步推理时，看到最精准、最少冗余的信息。当然实操中这件事情肯定是非常难，上下文窗口是有限的，塞太多东西 Agent 反而会"迷路" (lost in the middle)，塞太少又缺乏关键信息。怎么在这中间找到平衡，怎么在 Agent 运行过程中动态地裁剪和更新上下文，这就是 Context Engineering 主要关注解决的问题，根据每个项目和需求的情况又各有不同。

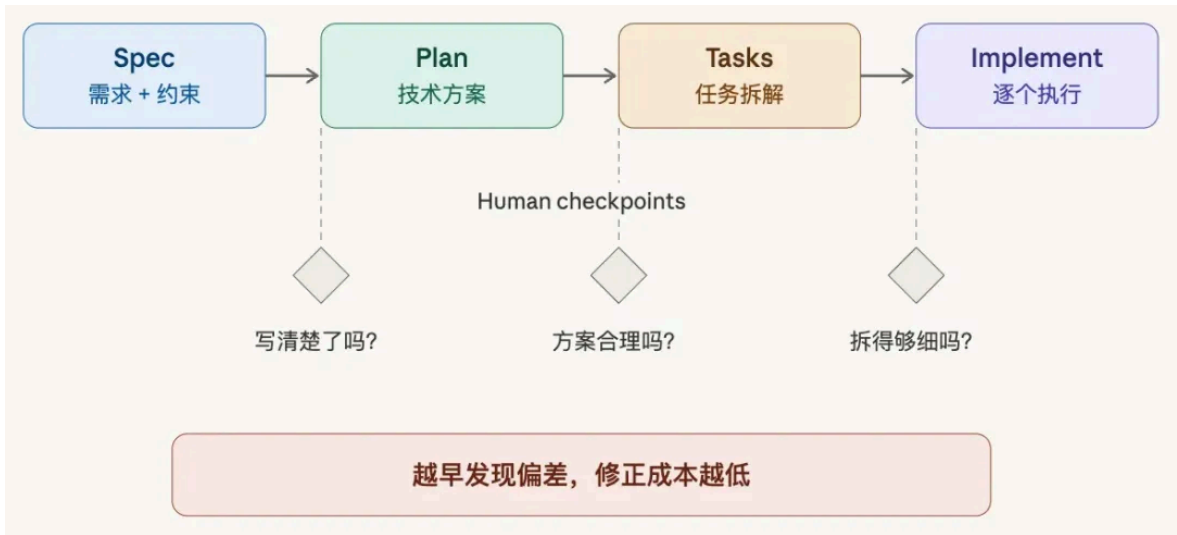
这里 Context Engineering 主要解决了「Agent 看到什么」，但没解决一个更根本的问题：**Agent 怎么知道你到底想要什么？**

1.2.3 Spec-driven Development: 先写契约，再让 Agent 动手

我相信很多人都有这种体验，我们跟 Agent 说"帮我实现一个 XX 功能"，它唰唰唰写了一大堆代码，你看了一下好像能跑，但仔细一看很多细节跟你预期的不一样。看起来像那么回事，但经不起检验。

问题出在哪呢？其实不是 Agent 笨，而是它擅长的是模式匹配，不是读心术。一个模糊的需求描述，Agent 会基于它训练数据中的模式去"猜"你的意图，有些猜对了，有些猜错了，而你往往到后面才发现哪里不对。

Spec-driven Development (SDD) 就是针对这个问题提出的方法。核心思路其实非常简单：**别急着让 Agent 写代码，先写一份 spec**。这个 spec 不需要是一份冗长的需求文档，而是一份清晰的契约，定义你要什么、不要什么、有哪些约束、验收标准是什么。然后让 Agent 基于这份 spec 去实现。GitHub 开源的 Spec Kit (<https://github.com/github/spec-kit>) 提供了一套比较成熟的工作流。



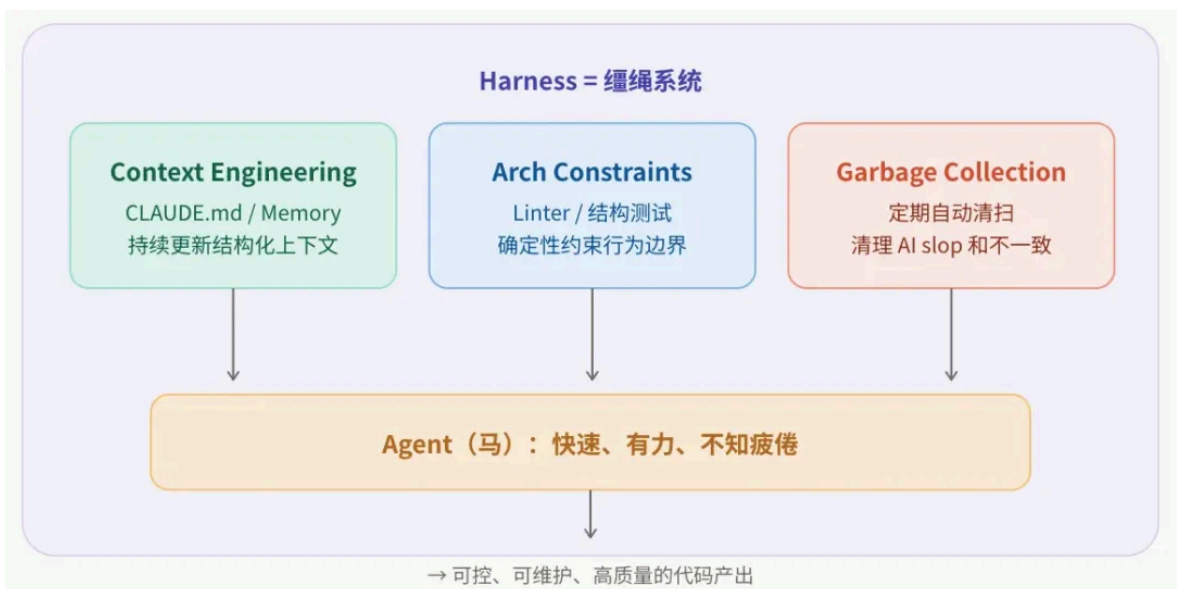
通过在每个阶段验证方向，使这种 gated workflow 看似变慢，实际大幅减少返工。

1.2.4 Harness Engineering：给 Agent 搭一个约束体系

如果持续用 Agent 维护项目一段时间，我们往往会发现：Agent 会复制仓库中已有的模式，包括不够好的模式。命名风格漂移、重复函数出现在不同角落、架构约束被悄悄突破，因为Agent 的吞吐量远超人类，熵的增长速度也远超预期。

这个概念在近期因 OpenAI 的一篇文章（<https://openai.com/zh-Hans-CN/index/harness-engineering/>）引发关注。他们用 3 个工程师、5 个月、完全零手写代码，构建了约百万行代码的内部产品。团队的核心工作不是写代码，而是搭建 Harness。

Octopus Deploy（<https://octopus.com/devops/continuous-delivery/harness-engineering/>）的比喻很贴切：Agent 是马，Harness 是缰绳。马本身快速有力，但没有缰绳就只会横冲直撞。



从 OpenAI 实践来看，一个 Harness 体系实际落地大致包含四个层面：

约束层：用机械化规则代替口头约定。 OpenAI 在实践中强制实施了严格的分层架构，跨层依赖不是靠文档告知，而是通过自定义 linter 和结构化测试来机械化检验。他们的经验是：如果一条架构规则值得写进文档，那它就值得用 linter 来强制执行。

文档层：把 AGENTS.md 当目录而非百科全书。 OpenAI 团队尝试过往 AGENTS.md 里塞满所有规则，发现效果很差——上下文是稀缺资源，当所有内容都被标记为“重要”时，等于什么都不重要。他们最终把 AGENTS.md 精简到约 100 行，只作为索引指向仓库中结构化的 docs/ 目录。用他们的话说：“给 Agent 一张地图，而不是一本千页手册。”

反馈层：构建“犯错→修复→沉淀”的飞轮。 业界之前对 Harness Engineering 提出过一个定义：每当 Agent 犯了一个错误，就工程化一个方案确保它不会再犯。OpenAI 团队就是遵循着这样的原则：当 Agent 卡住时，从不选择“再试一次”或人工介入，而是诊断“缺了什么能力”，然后让 Agent 自己把缺失的能力构建回仓库。每一次失败都转化为基础设施改进，而非一次性补丁。

清理层：自动化对抗熵增。 OpenAI 团队曾每周五都需要手动清理 Agent 生成的“AI 泥浆”，但他们很快认为这是不可持续的。后续他们最终的做法是把“黄金原则”编码为仓库级的规则，再用后台 Agent 定期扫描偏差、自动提交清理 PR。把技术债变成每天的小额偿还，而非积重难返后的大扫除。

1.3 让 Agent 替我学习

本节想讲的是我认为在当前时代即时收益最高的一个能力项：**高效获取各种最佳实践并高质量集成到个人工作流。**

1.3.1 「古法学习」行不通了

我其实一直很热衷进行资讯收集和个人笔记管理，比如通过 notion、obsidian 或者其他 web 工具构建所谓的数字花园，也比较喜欢通过技术博客、播客等渠道学习知识。但特别是今年随着大模型技术迭代的愈发快速，我发现这些「古法学习」已经行不通了。主要有以下痛点：

1. 现阶段技术迭代过快导致的信息量数以倍增，几乎不可能有足够的时间和精力去收集和钻研。
2. 技术范式完全没有稳定，并且各种分享良莠不齐，很难确认什么是真正有效的最佳实践，并且应用和试错的成本都比较高。

其实也很好理解，过往的每个领域（比如前端、后端、设计、测试等等）都主要是这个领域的从业者在为它添砖加瓦，但是大模型时代的 AI 应用领域却几乎是所有使用 AI 的人类一起在迭代，无分领域，无分文理，无分老少。特别当下又是 AI 应用爆发的初期，因此现今问题解决、方法演进和知识迭代的速度是过往难以想象的，当然这也是这个时代很让人兴奋和“痛苦”的地方。

1.3.2 让 Agent 帮我学，帮我用

针对上一节提到的痛点，我当下的态度是这样：既然我学习不过来了，那 Agent 帮我学习吧，既然我应用的比较慢，Agent 帮我应用。

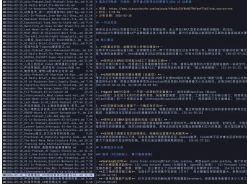
我希望把自己从信息采集的第一线撤出来，让 Agent 去做每日的资讯扫描和筛选，我只看它提炼后的”精华版“并在必要的时候做出反馈和记录。但不止于此，Agent 在采集过程中积累的知识会反哺到它自身的工作体系，使它在下一次被调用时比上一次掌握了更多业界前沿的优秀实践和解决方案。等到在实际工作中遇到项目需求或者技术挑战，帮我执行任务的 AI 小伙伴已经是一个经过最佳实践淬炼、武装了最新方案的 Agent。

目前我的知识管理主要是基于 Claude Code 结合 Skills 体系和 multi sub-agent 的方式来实现这个循环。



主要的 skill 介绍和示例如下：

Skill	层级	功能	示例/备注
ai-news	采集层	每日 AI 新闻聚合筛选。11 + 并行源（HN/HF/GitHub/Reddit/36Kr/量子位等），智能评分 0-10（跨源共振+3、突破+2），≥4	AI 日报 2026-03-18 信息简报，以下部分为自动提取的关键词 新增数据源 A (2 条) - DeepMind 发布 GPT-5 模型性能报告 (9/10) - AI - DeepState Rising 多模态融合 新增数据源 B 产品更新 (2 条) - Microsoft 发布 AI 助手 Copilot 5.0 版本，集成更多企业级应用 (8/10) - 商业科技, TransFormers 模型架构 新增研究论文 (2 条) - Online Experimental Learning (9/10) - LLM 在线持续学习新范式 - Reinforcement Learning for LLM Agents (8/10) - Grokking: 语言模型 (4/10), InCoder-3.0 工业代码模型 (4/10) - OpenAI Health Monitor Agent 评估 (5/10), Starline 360 (3/10) 新增开源项目 (1 条) - LangGraph 开源框架 异步编排 Agent (4/10) 新增社区动态 (1 条) - 腾讯 AI2 发布新模型 Hui G-1 (4/10) 新增书籍 (2 条) - 深度学习进阶：从基础 GPT 模型 (7/10) - Intermediate GPT 3.5 应用 Case (6/10) - AI 入门：从零基础到进阶 (5/10) 新增数据状态: 18/11 修正, Arxiv 图 GPT 训练失败 (来源 1 次), 数据源 B: /home/.llm/.ai/news_data/output/ai_news/2026-03-18.md

		分入选，seen.json 自动去重	
podcast-batch	采集层	批量转录+分析近期更新播客。自动刷新目录→提取近期→批量转录→逐批分析，支持 --days/--skip/--lang 参数，腾讯 ASR 10h/月免费额度	
web-collect	采集层	从指定网页/站点采集整理信息。目标确认→执行→清洗→格式化，支持 md/JSON/CSV 输出，记录采集时间和来源 URL	内嵌到其它各工作流内部调用
research	采集层	深度调研指定主题。多源采集+交叉验证，结构化报告（关键发现→详细分析→结论），所有声明标注来源	
ai-practices	提炼层	全自动提取可复用 AI 最佳实践。扫描 ai-news + 播客分析 + 网络搜索，严格准入（需 URL、≥50 字、可复用非一次性新闻），子 agent 生成避免同质化，自动去重+索引	
workspace-evolve	应用层	审计工作空间配置并建议优化。Full/Maintenance 两种模式，审计 CLAUDE.md/rules/skills/settings，对比官方+社区最佳实践，输出优化报告并实施	
doc-writer	应用层	撰写结构化文档。支持技术方案/指南/分析/总结等类型，素材收集→大纲→填充→审校，开头一句话摘要，Markdown 清晰层级	基于实际的汇报、分享等需要进行
podcast-script	应用层	从素材生成播客对话脚本并合成音频，方便通勤/跑步等时间快速了解今日最新信息。因为 doubao 中文语音能力极佳，这里是	

		使用的 coze 空间工作流 搭建后远程调用。	
tool-builder	应用层	构建实用工具/脚本。需求 分析→技术选型→实现→ 测试→归档，含 READM E、argparse/--help、完 整错误处理。帮我实现各 种各样的小工具。右图是 另一个工具，帮助我实现 快速连接各 linux 环境并分 屏布局。	
create-shortcut	应用层	生成 macOS 快捷指令(.sh ortcut)。Python 生成 plis t→签名→导入→验证，支 持条件/循环/变量等复杂 流程，18 种颜色+图标 库。也就是前文我提到的 快捷指令生成实践。	
generate-tests	应用层	生成全面测试套件。覆盖 单元测试、集成测试和边 界用例	内嵌到其它各工作流内部 调用

当然这套方案还在不停迭代中，远不能说是完善。但核心态度我觉得是很有意义的：在一个知识和范式都在高速变化的时代，与其追求"深入理解每一个东西"，不如追求"快速沉淀和验证每一个有价值的东西"。

02

碎碎念、思考和暴论

2.1 这实在是一篇太不 AI Native 的文章

撰写这篇文章其实花了我蛮多时间，但这本身就是最不 AI Native 的事情。因为知识整理和分享本来就是大模型非常擅长的事情，而且本文提到的绝大部分内容在我个人工作空间都有痕迹可以追寻。按理说当我有撰写文章的诉求，我的 AI 小伙伴就应该快速帮我呈现一篇美轮美奂的文章，其实我也确实这么尝试过，然后我就开始主要依靠自己进行撰写了.....

虽然很不 AI Native，但我倒也无意苛责自己，我只是等会儿得把这篇文章发给我的 AI 小伙伴仔细看看，用 1M 的 token 好好反思，你以后至少得比这篇文章写的强吧。claude code 你要是能干就干，不能干我就问问 codex 和 codebuddy 干不干。

其实还有一个很不 AI Native 的事情是，这篇讲 AI 的文章第一节居然推荐的是 MAC 电脑的工具链。我经常在想所谓的 AI Native 可能是什么，应该总不会还需要程序员苦哈哈熟悉一大堆工具、记忆一大堆快捷键。但就个人体感来说，claude code 仍然是我认为当下最好的 Agent 工具，所以我当下大部分时候就是围绕用好 claude code 在做事情，我不知道这是不是所谓「正确」的方向，但确实可能是当下对于程序员投入产出比最高的事情之一。

在我设想中，最理想的 AI Native 形态可能更像是小龙虾但不仅仅是小龙虾，因为我觉得目前即时通讯应用传输信息的形式还是太单薄了。我相信相关应用应该逐渐都会做出大的演进，也希望我司在这个战场能够一直领先。

2.2 与 AI 做好朋友

Andrej Karpathy 提出过一个概念叫 "Jagged Intelligence" (<https://x.com/karpathy/status/1816531576228053133>)，意思是 LLM 的能力分布跟人类完全不同。比如大模型可以解复杂的数学题，却分不清 9.11 和 9.9 哪个大；能识别上千种品种的花和狗，却数不对 "strawberry" 里有几个 r。虽然这些具体的例子在如今的主流模型上已经基本修复了，但它揭示的本质问题依然存在：大模型有些事做得惊人的好，有些事错得离谱，而且哪些行哪些不行并不显而易见。

在这个背景下 Karpathy 提出的解决方案就是「你可以随着使用逐渐建立起直觉」。持续使用是最好的老师，在与 AI 沟通一万句以后也许你就会发现很多困惑就自然而然迎刃而解。

我们有时候讲有了 AI 军团以后人人都是老板，也有时候说其实人类就是为 AI 搭建环境的服务员。但是我们现在换一种新的角度，与 AI 做无话不谈的好朋友，在愈发了解我的朋友以后，一起协作的过程中会不会更加的心有灵犀、游刃有余。

并且我在很多地方都看到过类似的观点，对于刚开始学习 AI 的同学来说也许不需要看那么多复杂的教程和实践，最简单高效的方法就是：「用最好的模型」+「持续与 AI 交流」

2.3 讲一下用 AI 的苦日子

再谈谈苦日子吧，比如在去年年中以前，我们如果希望使用顶级大模型和 AI 概念的前沿产品，需要先钻研的是什么呢。是要考虑如何合理越过被封禁的网络、是解决海外信用卡和手机号的限制、是想怎么薅点羊毛能够降低高昂的使用成本。感恩很多朋友和司内同学的积极讨论、分享，使那些问题都能够比较顺利的被解决，但想想很多时间花在了这些地方也很离谱。

但这不是最离谱的，最离谱的是在那些时光里面 AI 也不见得真正能让人提效。2025-ai-experienced-os-dev-study (<https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>) 这项实验就明确表示 25 年前期使用 AI 编程反而可能会降低效率。当然这个实验也有局限性在，但是其实也印证了当时对于 AI 的钻研带来的不一定是正收益。

更关键的是，大模型能力本身已经经历飞跃式的进步，大时代掀开了新的篇章，钻研 AI 带来的几乎一定是正收益。

AI 正当时，与诸位共勉

-End-

原创作者 | 李是希

感谢你读到这里，不如关注一下？📌



腾讯云开发者 

腾讯云官方社区公众号，汇聚技术开发者群体，分享技术干货，打造技术影响力交流社... >

1095篇原创内容

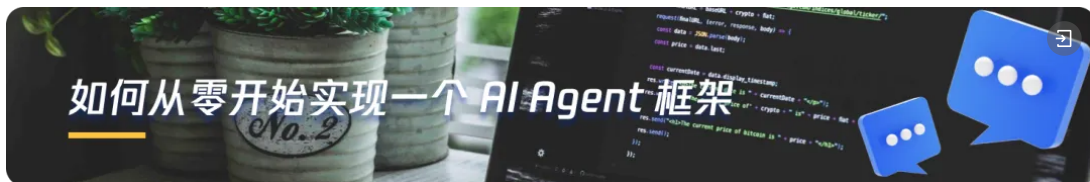
公众号



你对本文内容有哪些看法？同意、反对、困惑的地方是？欢迎留言，我们将邀请作者针对性回复你的评论，欢迎评论留言补充。我们将选取1则优质的评论，送出腾讯云定制文件袋套装1个（见下图）。4月20日中午12点开奖。



扫码领取腾讯云开发者专属服务器代金券！



赛博玄学，一键三连少一个Bug!

为好文章  点   收藏  点亮

腾讯技术人原创集 · 目录

<| 上一篇

办公Agent的CI/CD时刻到来了

下一篇 >

腾讯正式发布面向Agent设计的命令行工具：
CloudBase CLI V3

